

实验报告：让编程智能体进入可验证的修复循环

项目	内容
课程名称	系统开发工具基础
实验主题	智能体编程
题目编号	第10题
学号	25020007191
姓名	周御基
实验日期	2026-09-07

一、实验目的

1. 理解测试驱动（TDD）驱动编程智能体进行修复的循环。
2. 学会先编写失败测试，再向智能体说明目标、约束与测试命令。
3. 掌握人工检查 diff、撤销无关改动并复验测试的方法。

二、实验环境

- 操作系统：Ubuntu 25.04 (x86_64)
- 命令行工具：bash、git、python3.13、venv、pytest 9.1.1
- 其他软件：greetlab 包（复用第9题的 wheel 源码）、ruff 0.16.6

三、实验步骤与代码

步骤1：复制 q09 为 q10，添加失败测试

```
cp -r ~/lab/q09 ~/lab/q10
cd ~/lab/q10 && git init
```

新增 `test_cli.py`（要求：当 `--name` 只含空白字符时，`main` 应以 `SystemExit(2)` 结束）：

```

import sys

import pytest

import greetlab.cli as cli

def test_whitespace_name_exit_2(monkeypatch, capsys):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as excinfo:
        cli.main()
    assert excinfo.value.code == 2

```

步骤2：运行测试，确认失败并提交基线

```
~/lab/venv/bin/python -m pytest -q
```

输出结果：

```

F                                                                 [100%]
===== FAILURES =====
_____ test_whitespace_name_exit_2 _____
    with pytest.raises(SystemExit) as excinfo:
E       Failed: DID NOT RAISE SystemExit
_____ Captured stdout call _____
Hello,  !
===== short summary =====
FAILED test_cli.py::test_whitespace_name_exit_2 - Failed: DID NOT RAISE Syste...
1 failed in 0.07s

```

修复前 `main()` 直接 `print(f"Hello, {a.name}!")`，空白姓名仍输出问候并以 0 退出，测试如预期失败。

```
git add -A && git commit -m "baseline: failing whitespace-name test"
```

步骤3：向编程智能体说明目标、约束和测试命令

核心提示：

q10 的 greetlab 缺陷——`--name` 仅含空白字符时仍输出问候并以 0 退出；

目标：使 main 以 SystemExit(2) 结束；

约束：只修改实现、不改变正常姓名行为；

测试命令 `~/lab/venv/bin/python -m pytest test_cli.py -v`。

智能体改动：在 `cli.py` 的 `parse_args()` 后新增一行校验：

```
if not a.name.strip():
    p.error("name must not be blank")
```

步骤4：人工检查 diff，撤销无关修改并复跑测试

```
git diff
```

输出结果：

```
src/greetlab/cli.py | 2 ++
1 file changed, 2 insertions(+)
```

检查发现 `git add` 误入了 `src/greetlab/__pycache__/*.pyc` 缓存文件，将其加入 `.gitignore` 并从索引移除；确认 diff 仅含 `cli.py` 的 2 行改动。

```
~/lab/venv/bin/python -m pytest -q
```

输出结果：

```
. [100%]
1 passed in 0.02s
```

步骤5：正常行为回归并写 ai_log.md

```
~/lab/venv/bin/python -c "import sys,greetlab.cli as c; sys.argv=['sdt-greet','--name','25020007191']; c.main()"
```

输出结果：

```
Hello, 25020007191!
```

ai_log.md (不超过5行) :

1. 核心提示：q10 的 greetlab 缺陷----name 仅含空白字符时仍输出问候并以0退出；目标：使 main 以 SystemExit(2) 结束；约束：只修改实现、不改变正常姓名行为；测试命令。
2. 智能体改动：在 cli.py 的 parse_args() 后新增 if not a.name.strip(): p.error("name must not be blank")，并自测通过。
3. 人工验证：git diff 撤销无关改动（__pycache__/ 缓存已入 .gitignore 并移出索引），确认仅 cli.py。
4. 复测：pytest 通过。
5. 回归：sdt-greet --name 25020007191 输出正常，退出码0。

步骤6：提交修复

```
git add -A && git commit -m "Fix blank --name handling (exit code 2)"
git log --oneline
```

输出结果：

```
58c2615 Fix blank --name handling (exit code 2)
ba981fa baseline: failing whitespace-name test
```

四、实验结果

4.1 结果展示

- 基线提交下测试失败：DID NOT RAISE SystemExit (Hello, !)；
- 智能体修改后测试通过：1 passed in 0.02s；
- git diff 仅含 cli.py 的 2 行校验，无无关改动；
- 正常姓名回归输出 Hello, 25020007191!，未受影响；
- 修复已提交，工作区干净。

4.2 结果分析

- 以“失败测试→智能体修复→人工复核→复测”的闭环保证了改动可验证：目标行为被测试固化，智能体的修改不会失控。
- p.error() 由 argparse 内置实现，会打印错误信息并以退出码 2 结束，无需重复造轮子。

- 约束"只修改实现、不改变正常姓名行为"通过回归运行 `--name 25020007191` 得到验证。

五、实验总结

通过本次实验，我掌握了以下技能：

1. 编写失败测试并以此驱动编程智能体进入可验证的修复循环。
2. 撰写包含目标、约束和测试命令的智能体提示词，并核对 `git diff` 收敛改动范围。
3. 使用 `ai_log.md` 简洁记录提示词、改动与人工验证过程。

实验中的脚本文件：

1. `Scripts/q10/test_cli.py`
2. `Scripts/q10/src/greetlab/cli.py`
3. `Scripts/q10/pyproject.toml`
4. `Scripts/q10/ai_log.md`
5. `Scripts/q10/.gitignore`